Assignment 3

Due on November 24, 2025 (23:59:59)

The purpose of this assignment is to familiarize yourself with neural networks.

Classification of Urban Sounds using Neural Networks

For this assignment, you will implement a neural network with one hidden layer and Convolutional Neural Network(CNN) architecture to classify the sources of urban sounds.

Dataset

- You can download the dataset from this link .
- Dataset is split into 10 folds for k-fold cross-validation. You should instead do a simple train-test split, using the first 8/10s of the data for training and the remaining 2 as test data
- You will use UrbanSounds dataset that contains 8732 sound files. There are 10 seperate classes:
  - air_conditioner
  - car_horn
  - children_playing
  - dog_bark
  - drilling
  - engine_idling
  - gun_shot
  - jackhammer
  - siren
  - street_music

Feature Extraction

The dataset consist of short audio files. In order to use this data for training, You must firs transform this raw audio data into a format more suitable for training. You may create a Mel Spectrogram images and train using those images. You can also use other techniques but **you must specify what you did in your report**

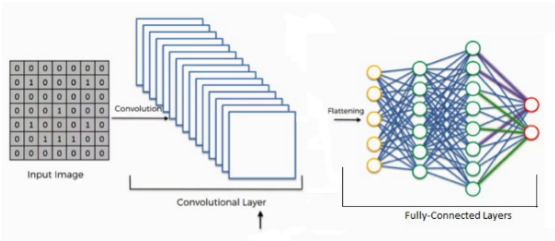

Figure 1: Multi layer neural network

Multi Layer Neural Network

In this part of the assignment, you have to implement multi layer neural network for classification. In other words your network consists of one input layer, n hidden layer(s) and one output layer. You will implement forward and backward propagations with the loss function and learning setting. Actually, you will implement a back-propagation algorithm to train a neural network.

In this step, you will implement the network given in Figure 1 and train the network feeding by the given training set. We can express this network mathematically as:

$$O_i = w_{ij}xj + bi$$

As a loss function, you will use the sum of the negative log-likelihood of the correct labels. Write a Python function to compute the loss function. Then you will update network parameters w and b to minimize the loss function using gradient descent algorithm. You will implement a function that computes the derivative of the loss function with respect to the parameters. To make sure your function is correct, you must also implement numerical approximation of gradients.

Write a function to minimize your cost function using mini-batch gradient descent. You should try different learning rates (between 0.005—0.02) and batch sizes (between 16—128). Make a table to show the learning performance for each setting you tried.

Finally, you will visualize the learned parameters as if they were images. Visualize of each set of parameters that connect to $O_0, O_1, ..., O_n$.

Convolutional Neural Network

In this part of the assignment, you have to implement a Convolutional Neural Network(CNN) for classification (Figure 2). In other words, your network consists of n convolutional layers, m fully-connected layer(s), and one output layer. You will implement forward and backward propagations with the loss function and learning setting as explained in the previous section. Actually, you will implement a back-propagation algorithm to train a neural network.

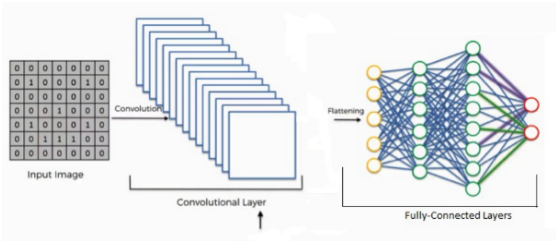

Figure 2: An example convolutional neural network

Training a network

- You should determine the number of units in your convolutional layers and fully connected layers.
- You should determine the convolutional layer's width, height, and depth parameters.
- You should determine batch size as you learned in class.
- You should determine a learning rate for your gradient descent method.
- Remember, the learning rate parameter may be a problem (too big - may not converge, too small - very slow convergence). For this reason, you can define a learning rate decay parameter. You will start with a learning rate value and after each epoch, you will reduce the learning rate by multiplying it by a decay rate. This operation can deal with the mentioned problem.
- You can use different activation functions: Sigmoid, tanh, ReLU, etc.
- You can use different objective functions: Cross-entropy error, the sum of squared error (SSE), etc.
- You can control your implementation by plotting loss. You can see if it converges or if it needs a different parameter setting.
- **You should discuss about your each experiment in the report. Comment about their effects.**
- Save your trained models to use later in test time.

Implementation Details:

For implementing CNN, you can utilize libraries from the PyTorch (https:// pytorch.org/) and you can use transfer learning which means you can fine-tune a pre-trained CNN model for the classification.

Important Notes About Assignment:

- You can use a table for reporting your results.
- You will implement a single-layer neural network(a neural network with only has an output layer) and run experiments on the dataset. You will change parameters (activation func., objective func. etc.) and report results. **Obligatory**
- Then you will change your architecture and use a network that contains two hidden layers. Repeat the same experiments and comment on the results. Obligatory
- You will implement a convolutional neural network that contains one convolutional layer and one fully connected layer. You'll change the mentioned parameters (unit number in the hidden layer, activation function, etc.) and report the results. Obligatory
- Then you'll change your architecture and use a network that contains two convolutional layers and two fully connected layers. Repeat the same experiments and comment on the results. Obligatory
- You can also try different layer sizes (convolutional or fully connected) for your normal neural network or CNN model. But, you should not restrict yourself to the obligatory models above (especially for CNN), as you try different combinations with layer size and other parameters, your chance to get full points from the analysis report increases.
- You have to comment about results and parameters' effects on different values (learning rate, batch size, layer size, hidden neuron size in the layers, etc...).
- Your implementation should be reproducible, in other words, do not write separate code for each architecture. If you use n layers, your method should create a n-layer network and learn the classifier. (For MLP and CNN)
- Comment your code with corresponding mathematical functions and explain what is going on in your code in the code scripts.
- You should also compare your classic neural network and convolutional neural network with respect to the accuracy, parameter (weight size) size, and training-test error curve plots and you must state every experiment you accomplish and related proper explanation/ conclusion about why you obtain such a result in your analysis report in order to get full points on the analysis report.

Submit

You are required to submit all your code (*all your code should be written in Jupyter notebook* long with a report in ipynb format (should be prepared using Jupyter notebook). The codes you will submit should be well commented. Your report should be self-contained and should contain a brief overview of the problem and the details of

your implemented solution. You can include pseudocode or figures to highlight or clarify certain aspects of your solution.

Finally, prepare a ZIP file named **name-surname-pset2.zip** containing

- reportcode.ipynb (".ipynb" file including your code and report)
- code/ (directory containing all your codes as Python file .py)

Grading

- Code (60):30 points for multi-layer neural network implementations, 20 points for convolutional neural network implementations, and 10 points for other code parts.
- Report(40): Analysis of the results for classification:
- Bonus(10): Using more than one method for feature extraction

  Note: Preparing a good report is important as well as the correctness of your solutions! You should explain your choices and their effects to the results. You can create a table to report your results.

Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your classmates about the given assignments, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else.

References